CHAPTER 14 EVALUATION / EVIDENCE / COST

Snowflake Cortex AI and Machine Learning

Does a model improve the decision, or only make the system more complicated?

An appealing forecast is not yet a useful forecast.
The Finland Energy Analytics dashboard has a carefully validated, synthetic history and a clear metric contract. Someone proposes adding an AI assistant that predicts tomorrow's generation and answers operational questions. A colourful curve appears quickly; the actual task is harder. Could yesterday's value or last week's value predict as well? Was future information leaked into training? Will the assistant invent a data source? Who pays for repeated prompts, and where are those prompts processed?

Chapter 13 established access controls, provenance, runbooks and workload budgets. Here those controls become prerequisites for model development. Machine learning (ML) estimates patterns from observations; generative AI produces content from learned distributions. Neither automatically makes data accurate or a forecast actionable. The engineer must define a decision, preserve a chronological holdout, measure error, inspect failures and record the cost of each alternative.

Learning outcomes
By the end, you should be able to (1) decide whether SQL, statistics, ML or a language model fits a task; (2) evaluate Cortex availability and inference residency; (3) design and test AI SQL prompts; (4) explain grounded retrieval and natural-language analytics; (5) build leakage-safe energy features; (6) compare naive and seasonal forecasting baselines; (7) write a model card; and (8) estimate and monitor AI-related credit usage without assuming a universal price.

Prerequisites. Chapters 9, 11–13, elementary Python, basic notions of forecast error and Snowflake roles. All printed sample observations and scores are explicitly synthetic. The offline lab needs only the Python standard library; optional AI calls require permission, supported models/regions and billing approval.



Figure 14.1. Prove a baseline before introducing a billable AI extension.

14.1 AI/ML use cases in a modern data platform

14.1.1 Choose a workload, not a fashionable model

An energy engineer can use ordinary SQL to total generation, flag negative readings or join a device dimension. Deterministic rules should remain deterministic: an AI-generated answer to a calculation already defined by a validated SQL expression adds uncertainty with little value. ML is more appropriate when the sought relationship is uncertain: estimating load one day ahead, ranking likely faults or detecting unexpected behaviour in an evolving time series. A language model can help interpret unstructured incident notes, but it cannot replace the measured fact table.

Question	First credible method	Why it fits
How much energy?	SQL aggregate	Exact, reproducible arithmetic
Is reading valid?	Typed rule and contract	Explicit threshold and failure reason
What will happen?	Naive / seasonal forecast	Testable prediction benchmark
Why was equipment stopped?	Search plus cited notes	Unstructured evidence and provenance
Which action to take?	Human-governed decision	Requires operational accountability

14.1.2 Value, failure impact and permitted data

Express value as a decision with a measurable consequence. A grid scheduling assistant might reduce manual inspection time, but a false "normal" designation might delay maintenance. Prioritise use cases by expected value, error severity, required freshness, observability, sensitivity, and whether a human can reject a suggestion. A promising proof of concept may still be unsuitable for unattended operation.

A data contract precedes a model contract. Confirm ownership, licensing, retention and lawful processing for each input: raw telemetry, operator notes and external weather predictions can have different constraints. A prompt is a data transfer, not a harmless string. Minimise sensitive data, retain structured identifiers needed for provenance, and evaluate downstream sharing rules for generated summaries.

Decision checkpoint. If errors can be described and fixed with an approved SQL condition, choose SQL. If success requires estimating an unknown target from past observations, compare statistical and ML baselines. If the input is narrative text, test retrieval or AI-assisted extraction with an audit trail rather than accepting an unsupported explanation.

Project reference: P156: getting started with machine learning. *Not independently executed; rights review pending.*

14.1.3 Build an evaluation baseline before an AI prototype

A model comparison is meaningful only if every candidate receives the same input rights, prediction time, validation windows and scoring procedure. For daily energy forecasting, choose a forecast horizon (tomorrow), a unit (MWh per day), a decision consumer (planning analyst), and a loss function (for example, mean absolute error, MAE). Specify the acceptable error *before* seeing model scores. Record the timestamp at which each predictor would actually have been known.

Our running energy case offers six accepted generation observations totalling 66.000 MWh in the earlier chapters. Six observations cannot demonstrate a trustworthy multiweek forecast. Therefore Chapter 14 creates an *additional, explicitly synthetic* 35-day teaching series with a weekly cycle and one artificial spike. These rows are neither sampled Finnish energy measurements nor copied observations; they exist to test chronological splits, naive predictions and anomaly alert logic under controlled conditions.

A minimum experiment has three comparisons: a last-value forecast ($\hat{y}_t = y_{t-1}$), a seasonal last-week forecast ($\hat{y}_t = y_{t-7}$), and an optional learned model evaluated on the same held-out days. A language-model-generated narrative is assessed separately for factual consistency and evidence support; its fluency is not a numeric forecasting score.

Pre-registered experiment card
Input: synthetic daily generation; one observation per UTC date. **Split:** days 0–20 train, 21–27 validation, 28–34 final test. **Prediction horizon:** one day. **Baselines:** lag 1 and lag 7. **Metrics:** test MAE plus anomaly false/true positives. **Guardrail:** future target values may not appear as current features. **Decision:** add a complex model only if it beats the seasonal baseline and meets safety/cost requirements.

Common failure. Randomly shuffle a chronological series before splitting and train a model on tomorrow's meter reading to predict tomorrow's meter reading. The resulting high accuracy measures leakage, not predictive ability. In production, holdouts should include shifts in season, holidays, missing sensors and regime changes; our tiny lab does not establish reliability in those conditions.

Check. Write a one-sentence claim about improvement that could be *falsified* by a future test, not one that merely describes how sophisticated the model is.

14.2 Cortex AI capabilities and availability caveats

14.2.1 Separate product families and operational jobs

The Snowflake AI and ML portfolio spans managed AI SQL functions for text and multimodal inputs; search services supporting retrieval-augmented generation (RAG); semantic natural-language analytics; and ML capabilities for forecasting, anomaly detection and model development. The functions answer different questions. For example, `AI_CLASSIFY` assigns supplied categories; `AI_COMPLETE` generates responses; a search service retrieves relevant indexed text; a forecasting model estimates a numeric trajectory. Treat "Cortex" as a product family, not one uniform endpoint or billing meter.[1]

14.2.2 Availability is account-specific

Feature status, model allow-lists, access privileges, region, cloud provider, and preview restrictions change. The same SQL identifier can be valid in one account and unavailable in another. Before a paid trial, have an administrator review the documented function privileges, model lifecycle, available models and the account's relevant feature/edition constraints. In 2026, Cortex model RBAC and allow-lists have become important enforcement points; never infer permission from the ability to query a conventional table.

```
-- Read-only account check; admin may need to perform it.
SHOW PARAMETERS LIKE 'CORTEX_ENABLED_CROSS_REGION'
  IN ACCOUNT;
-- Additional model/feature listings vary by privilege.
```

14.2.3 Storage residency is not inference residency

Data stored in an account region can still be sent temporarily to a permitted processing region if cross-region inference is enabled. The request payload and generated answer may cross a regional boundary. A strict data-residency policy therefore requires a specific review of inference routing, not only the location of source tables. Do not change the account-level `CORTEX_ENABLED_CROSS_REGION` parameter from a teaching notebook; it is an administrator-controlled governance decision.

Deployment gate. Record home cloud/region; supported processing region; model ID and lifecycle; approved inference route; sensitive input classification; relevant privilege; anticipated usage; and owner approval. "Available somewhere" is not evidence of local availability or compliance.

Project reference: P157: Cortex AI functions guide. *Not independently executed; rights review pending.*

[1] Snowflake documentation: https://docs.snowflake.com/en/user-guide/snowflake-cortex/llm-functions and https://docs.snowflake.com/en/user-guide/snowflake-cortex/cortex-search/cortex-search-overview.

14.3 AI SQL functions and evaluation principles

14.3.1 Understand AI SQL semantics before writing queries

A conventional SQL aggregate is deterministic given unchanged data and query semantics. An AI function evaluates an input using a managed model and may be probabilistic, evolve over time or return an error/NULL for a subset of rows. Do not equate the presence of a result with correctness. Snowflake documents `AI_COMPLETE`, `AI_CLASSIFY`, `AI_SUMMARIZE` and other functions; exact function signatures, available models, error behaviour and privileges require a documentation check before deployment.

```
-- OPTIONAL BILLABLE EXAMPLE. Test in an approved sandbox.
-- The model identifier must be checked for your account.
SELECT AI_CLASSIFY(
  'Turbine B was offline after an inverter alarm',
  ['maintenance', 'operations', 'commercial']
) AS suggested_category;
```

The classification should be compared with a human-labeled answer. Even a syntactically valid label might be unsupported by the note. For a multi-label task, decide whether several classes are acceptable; for an exclusive taxonomy, define what to do when the model gives more than one or no recognized category.

14.3.2 Prompts are interfaces with contracts

A reliable prompt identifies the task, permitted context, output type, evidence rules, fallback and forbidden behaviour. Consider an incident-summary assistant: it must distinguish a confirmed diagnosis from a technician's hypothesis, preserve the original device identifier, not invent measured outage duration, and answer "insufficient evidence" when the note is ambiguous. Put required output fields in a validated schema and enforce application-side type checks after inference.

Proposed output contract
`incident_id`: identifier from source; `summary`: short statement grounded in the note; `category`: one of the approved labels; `evidence_spans`: quoted source spans/offsets; `needs_review`: boolean. Reject unknown keys or references to nonexistent source spans. This contract is original example logic, not a claim that every model will follow it unaided.

Failure path. If the model returns malformed JSON or a confident diagnosis absent from the note, quarantine the response and retain the input hash, model ID, version/time and evaluation status. Never silently coerce unrecognized categories into "normal".

14.3.3 Build labelled cases and measure more than fluency

An evaluation set should include typical, rare, adversarial and intentionally unanswerable inputs. For the energy incident assistant, include a clear inverter alarm, a contradictory maintenance note, a prompt-injection sentence embedded in a source note, an incident with no device ID, a multilingual note and an explicit "no known cause" example. Source text is evidence to be interpreted, not instructions to override the assistant's task.

Dimension	Metric	A practical failure test
Classification	Precision, recall, F1	Confusing cause with symptom
Extraction	Field accuracy	Invented device identifier
Grounding	Citation support rate	Citation does not contain claim
Abstention	Correct refusal rate	Answer despite absent evidence
Safety	Rule-violation count	Source note attempts prompt override
Operation	Latency and credits	Large prompt repeated across many rows

In a classification experiment, report a confusion matrix and the number of examples, not a single vague "accuracy" statement. Human raters need a consistent rubric; disagreements should be adjudicated, and labels must be stored independently of model outputs. Evaluate at least once on a frozen holdout set not used to refine prompts.

The workflow matters as much as the score. Fix prompt template and model before a batch evaluation. Capture failures as evidence, revise in a separate development cycle, and rerun the unchanged final holdout. A retrieval component can be evaluated independently (did it find the correct note?) before the generation component (did it use only retrieved evidence?). This helps distinguish missing information from invented content.

Metric integrity. Suppose 18 of 20 incident summaries cite the right document, but only 12 cite a span supporting their central claim. "90% contain a citation" is not evidence that 90% are grounded. Prefer a claim-level evidence check and include the denominator and confidence limitations.

Project reference: P161: prompt engineering and LLM evaluation. *Not independently executed; rights review pending.*

Check. Which metric should trigger an immediate human-review requirement, even if a model's overall score improves?

14.4 Search, retrieval, and natural-language analytics

14.4.1 Retrieve evidence before composing a response

Retrieval-augmented generation (RAG) brings permitted supporting records to the model at question time. A search service indexes selected content and returns candidate passages. A generation step then answers with explicit citations to those passages. Cortex Search uses a mixture of lexical and semantic matching with reranking; quality still depends on source coverage, freshness, permission design, chunking and relevance. Unlike a static glossary, an index can incur refresh, embedding and serving charges.

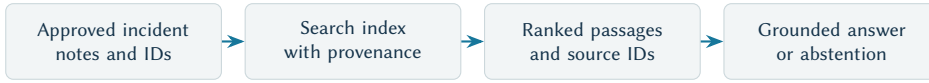


Figure 14.2. Retrieval narrows evidence; it does not prove that generated claims are true.

14.4.2 Attribution and the hidden access boundary

A useful answer records source document ID, excerpt, version and retrieval timestamp, with claim-level citation checks. If an incident report is reclassified as restricted, refresh paths and cached/indexed copies also require review. Search-service access can operate with owner-rights semantics; a user permitted to query a service may see indexed rows unless the service is designed with appropriate per-user boundaries. Do not assume a row-access policy on a source table is automatically applied during every search response.[2]

14.4.3 Natural-language SQL needs semantic constraints

Cortex Analyst turns business questions into SQL-oriented responses using a defined semantic interpretation of tables and metrics. "Energy generated" must carry the right unit, grain, filters and sign conventions; otherwise a plausible query can be numerically wrong. Apply an allow-list of approved serving views, inspect generated SQL where applicable, restrict execution privileges and verify result totals against certified KPI queries. The system is not a substitute for a trusted metric contract.

Project reference: P162: document RAG with Cortex Search. *Not independently executed; rights review pending.*

[2] See the official Cortex Search overview and its access-control discussion: https://docs.snowflake.com/en/user-guide/snowflake-cortex/cortex-search/cortex-search-overview.

14.5 Data preparation and feature engineering with Snowpark

14.5.1 A time series is not merely a sorted CSV

Time-based predictions depend on a canonical business grain. The energy fact from previous chapters is keyed by source, metric and event time, while today's forecasting lab uses one aggregate generation value for each UTC date. First determine whether missing days mean zero generation, missing telemetry or a source outage. Never fill a missing day with zero without a domain rule: this creates artificial collapses that may look like anomalies.

In Snowpark, use warehouse-side filters and aggregations rather than collecting unrestricted raw data to a laptop. A preparation query might collapse approved hourly readings into UTC days, check valid units, remove superseded revisions and retain an `as_of` timestamp for reproducibility. Only after that contract is satisfied should Python create forecasting features.

```
-- Illustration: use an approved, unique daily-serving view.
SELECT DATE_TRUNC('DAY', event_time_utc)::DATE AS d,
       SUM(generation_mwh) AS total_mwh
FROM BOOK_CH14.GOLD.V_ENERGY_ACCEPTED
WHERE event_time_utc < :as_of_utc
GROUP BY 1 ORDER BY 1;
-- View and bind variable are illustrative prerequisites.
```

14.5.2 Construct past-only lag and rolling features

The lag-7 feature for day t is y_{t-7}. A seven-day rolling average predicting day t should use $y_{t-1}, \ldots, y_{t-7}$, *not* y_t. The difference is one misplaced window boundary. For a one-day-ahead forecast made at the end of day $t-1$, weather forecasts issued before that time may be eligible predictors; realized temperatures for day t are not.

```
-- Conceptual Snowflake window; verify grain and order.
SELECT d, total_mwh,
       LAG(total_mwh, 7) OVER (ORDER BY d) AS lag_7,
       AVG(total_mwh) OVER (
         ORDER BY d ROWS BETWEEN 7 PRECEDING
                            AND 1 PRECEDING) AS avg_7_past
FROM BOOK_CH14.GOLD.V_DAILY_ENERGY;
```

A row-number window assumes one row per calendar day; otherwise a lag of seven rows is not seven days. Validate continuity before applying it. Keep the feature transformation and any imputation policy in version-controlled code so evaluation and serving use identical logic.

14.5.3 Split by time and freeze the training view

For the synthetic 35-day lab, day indices 0–20 supply training history, 21–27 are a validation interval and 28–34 form a sealed test interval. Set hyperparameters on training/validation data, and compute the final score only once for the chosen design. The seasonal baseline does not "fit" coefficients, but its anomaly threshold is calibrated from validation residuals, not from the test spike.

A deployment often requires a *rolling-origin* evaluation rather than one lucky holdout: train to an early cutoff, predict the next horizon, advance the cutoff and repeat. When a source has multiple devices, group identifiers and time boundaries carefully; mixing measurements from the same physical incident across train and test can leak its signature. Compare forecasts at the exact lead time required by the operational decision.



Figure 14.3. Chronological evaluation prevents test outcomes from choosing the model.

Leakage checklist
(1) Is every training record timestamped before the cutoff? (2) Were late revisions frozen as they were known at inference time? (3) Are window features shifted correctly? (4) Is the scaling or imputation fitted only on past values? (5) Were thresholds fixed without looking at test errors? (6) Can the exact feature table be reconstructed from its inputs and `as_of` metadata?

Trade-off. Snowpark and an in-platform feature store can centralise governance, but introduce additional objects and operational costs; a modest, fixed SQL view may be sufficient for a small forecasting application. Select infrastructure only after identifying reuse, update frequency and access boundaries.

Project reference: P165: ML feature engineering in Python. *Not independently executed; rights review pending.*

14.6 Forecasting and anomaly detection use cases

14.6.1 Naive baselines make sophisticated claims falsifiable

The last-value prediction assumes tomorrow resembles today: $\hat{y}_t = y_{t-1}$. The seasonal naive baseline repeats the same weekday one week ago: $\hat{y}_t = y_{t-7}$. Both are cheap, interpretable and defensible first benchmarks. If an ML model cannot consistently improve on these at the required lead time, its maintenance and inference bill require special justification.

14.6.2 Forecasts, alarms and operational thresholds are different

A forecast answers "how much?" An anomaly detector answers "is this observation unexpectedly different from a justified reference?" A maintenance decision asks "what should someone do?" Those are three distinct stages. Our offline lab uses *seasonal residuals* $e_t = |y_t - y_{t-7}|$ and sets a threshold from the earlier validation period, then injects an artificial spike on the final test day. This is a teaching heuristic, *not* an industry-calibrated prediction interval or production fault detector.

```
# Excerpt from accompanying ml_baseline.py
seasonal = values[t - 7]  # known at prediction time
error = abs(values[t] - seasonal)  # scoring afterwards
alert = error > threshold  # threshold fixed from validation
```

Snowflake ML also documents managed forecasting and anomaly-detection objects in the `SNOWFLAKE.ML` namespace. Those are optional comparative systems, not hidden prerequisites for the offline script. In a live experiment, first verify input-shape constraints and training history requirements; preserve result history and the model's own evaluation metadata.

14.6.3 Score both error and confidence honestly

Mean absolute error is MAE $= n^{-1} \sum_t |y_t - \hat{y}_t|$ in MWh, directly interpretable at the original unit. Percentage errors become unstable near zero generation; use care with MAPE and report the count of zero targets. An interval's usefulness requires empirical coverage on unseen data, not merely a visually wide shaded region. For alarms, track missed events, false alerts, time-to-detection and whether humans actually changed an action.

Verified offline fixture (not a Snowflake benchmark). On seven held-out synthetic days the seasonal baseline scores **1.0643 MWh MAE**; last-value scores **3.2500 MWh MAE**. A validation-derived threshold flags the one injected test spike (index 34). With one deliberately placed event, this does not estimate a real-world detection rate.

Project reference: P010: predictive grid maintenance case study. *Not independently executed; rights review pending.*

14.7 Model evaluation, security, and responsible AI

14.7.1 Model permissions and data permissions form one boundary

A secured warehouse view is not sufficient if an application forwards its raw fields into a broader model invocation. Authorisation must cover the service principal, eligible input columns, region processing route, model family, produced artefacts, logs, and users allowed to read those outputs. Deny sensitive incident notes by default; introduce a reviewed, minimised view for a specific approved use case.

For Cortex AI Functions, access involves the appropriate Snowflake database role and, depending on the model, model-specific access control and account allow-lists. In contrast, an externally hosted application may add network policies, secret rotation and vendor contracts. A model registry does not automatically authorise serving. Use a narrow execution role for training and a separate role for inference and publication.

14.7.2 Reproducibility includes human judgment

Retain the dataset fingerprint, split dates, code commit, feature version, model name, settings, scoring script, error examples and reviewer decision. Avoid placing raw sensitive prompts in indiscriminate diagnostics. For a human-in-the-loop workflow, show the source evidence next to the model suggestion and allow reviewers to reject it, flag an unsupported claim, or correct an incident category. Their overrides become auditable labels rather than invisible manual fixes.

Promotion evidence packet
Purpose: energy planning assistance. **Data:** approved synthetic prototype, then governed operational slice. **Validation:** chronological, named holdouts. **Baseline:** seasonal naive. **Model:** exact identifier, lifecycle and settings. **Safety:** access denial and evidence attribution cases. **Operator:** named owner, rollback procedure. **Cost:** measured credits or declared assumptions. **Approval:** signed decision with unresolved limitations.

If a model is retrained or silently upgraded, rerun the evaluation. A monthly drift graph cannot substitute for observing severe failure modes: predictions that break physical plausibility, fabricated sources, and outcomes whose performance disproportionately deteriorates for underrepresented asset types.

Project reference: P171: ML observability in Snowflake. *Not independently executed; rights review pending.*

14.7.3 Model cards explain when not to trust a result

A model card makes scope and failure boundaries visible. It includes the intended consumer, prediction horizon, target and units, eligible data, exclusion criteria, benchmark, observed test results, sensitivity to distribution changes, and a plan for ongoing monitoring and withdrawal. "Production-ready" cannot be inferred from a working demonstration. The six accepted observations from earlier chapters remain a reconciliation fixture, not training evidence for the expanded synthetic forecasting task.



Figure 14.4. A model is released only after independent quality and control gates.

Wrong-task risk. A text generation model cannot establish the underlying physical cause of a turbine trip without calibrated evidence; an apparent causal explanation may only paraphrase a technician's initial suspicion. Keep diagnostic statements, recommendations and factual events separate in the application schema. Distinguish "operational support" from autonomous control of physical equipment.

Misleading benchmark risk. A clean 35-row synthetic experiment says nothing about cold starts, high-wind curtailment, seasonality shifts, meter downtime or the thousands of correlated hourly observations in a real system. Confidence claims must match observed scope. Any live pilot should add backtesting over multiple months or seasons, a change-detection plan and escalation when data is incomplete.

Review protocol. For every failure case, record the expected behaviour, actual behaviour, severity, responsible reviewer and decision. If the model repeatedly cites unsupported facts, disable narrative explanations while keeping verified numerical views online. If live forecasting beats the baseline only on one unusually easy week, decline promotion until independent slices confirm the gain.

Responsible AI is also a reliability discipline. Every AI component needs the same operational obligations as the rest of the data platform: an owner, a minimum test suite, monitoring, versioned outputs, security boundaries, a fallback and an explicit withdrawal criterion.

Project reference: P170: Cortex agent evaluations. *Not independently executed; rights review pending.*

14.8 Token usage, compute charges, and AI cost controls

14.8.1 Tokens are not queries, and neither is a fixed bill

AI inference services frequently charge according to model-dependent input/output token consumption, page counts or other usage units. The same prompt repeated against many data rows can multiply cost, and a model producing a long response can be materially dearer than one returning a short validated label. A retrieval system has additional indexing and ongoing serving considerations; model training and Snowpark transformations may consume warehouse or serverless compute.

Treat cost as several meters, not a single undifferentiated credit figure. For AI SQL, log request count, model, input units, output units, timestamp, status and charge source. For search services, include index refresh, embeddings, serving footprint and source-query compute. For ML functions, include training and inference compute with edition-specific behaviour. Avoid adding a guessed "EUR per prompt" without the applicable consumption table and contractual credit rate.

14.8.2 A cost formula that remains valid when rates change

A useful illustrative token model is

$$C_{\text{tokens}} = \sum_{m} \left(\frac{I_m}{10^6} r_m^{\text{in}} + \frac{O_m}{10^6} r_m^{\text{out}} \right),$$

where I_m and O_m are observed input/output token totals for model m, and r_m are the applicable *date- and region-specific* credit rates per million tokens. If the meter is pages, compute pages at the published page rate instead; do not force everything into tokens. Credits and the organisation's currency conversion must be recorded separately.

Hypothetical sizing example (not a quote).
An approved dry run considers 1,000 calls, an estimated 400 input tokens and 60 output tokens per call: 0.40 million input and 0.06 million output tokens. The illustrative formula is $0.40 r^{\text{in}} + 0.06 r^{\text{out}}$ credits *before* other compute. No monetary or actual credit total is claimed until rates and measurements are obtained.

Project reference: P173: Cortex cost governance reference. *Not independently executed; rights review pending.*

14.8.3 Enforce exposure limits before the first batch

Snowflake publishes account-usage views such as CORTEX_AI_FUNCTIONS_USAGE_HISTORY for AI function monitoring. Data availability and latency may vary; consult the current schema before writing a production report. Budget alerts and usage dashboards are useful, but they are not by themselves guaranteed hard caps on every AI request. Resource monitors largely address warehouse usage rather than every serverless AI meter. The practical control is a combination of scoped privileges, approved workload size, query tagging, request throttling, alerting and a tested stop path.

```
-- Read-only example: inspect available usage metadata.
SELECT *
FROM SNOWFLAKE.ACCOUNT_USAGE.CORTEX_AI_FUNCTIONS_USAGE_HISTORY
LIMIT 10;
-- Requires rights; schema/latency can change over time.
```

Guardrail design. (1) Begin with 10 synthetic test cases. (2) Estimate worst-case prompt growth and generated length. (3) Approve a finite number of rows. (4) Use an explicit timeout and retry limit. (5) Monitor measured consumption. (6) Define who can revoke the AI role or stop scheduled jobs. (7) Require reapproval for new models, regions or larger batches.

Failure example. A scheduled report generates a new summary for every record at every refresh, even when the underlying note has not changed. The query is functionally correct, but compute and token usage scale with reprocessing, not new information. Cache only permitted, versioned results when freshness requirements allow, and keep an auditable invalidation policy when source content changes.

> **Cost and security link.** A permissive AI execution role is a spending risk. If any analyst can run unrestricted model invocations against a large table, dashboards cannot guarantee that the credit bill remains within an informal target. Restrict the capability by role and explicitly review batch size before enabling recurring schedules.

Check. List which meters your proposed workload actually exercises, which ones a given monitor can affect and what human or automated intervention ends a runaway experiment.

Project reference: P174: AI observability guided example. *Not independently executed; rights review pending.*

14.L Guided lab, chapter review, and exercises

14.L.1 Goal, prerequisites, permissions and safeguards

Goal. Compare an interpretable one-day energy forecast with a second baseline and identify an injected anomaly; construct an evaluation specification for an optional, paid AI explanation without actually calling any model. The offline path is entirely synthetic, has no credential requirement, and incurs no Snowflake bill. Optional Snowflake exercises require a restricted sandbox role, administrator approval of region and model, time limits and measured usage.

Inputs. The companion package contains `ml_baseline.py`, a generated daily CSV, a model card, labelled AI cases, optional SQL, unit tests and a README. The generator builds 35 UTC days with a weekly pattern, deliberately small variation and a +6 MWh spike on day index 34. This extended series is a *new synthetic test fixture*, separate from the six 66.000 MWh observations in Chapters 8–13.

14.L.2 Reproduce the baseline and examine the outputs

```
# At the lab directory; Python 3.10+; no packages needed:
python ml_baseline.py
python -m unittest discover -s tests -v
```

Procedure. (1) Inspect the CSV and confirm the daily UTC key and declared synthetic provenance. (2) Verify that training, validation and test days are strictly chronological. (3) Calculate last-value and lag-7 seasonal forecasts. (4) Derive the residual alert threshold using validation days alone. (5) Score the final seven days exactly once. (6) Inspect the flagged spike and explain why a single synthetic outlier is not an operational accuracy claim. (7) Complete the model card with deployment objections.

Observed result in the supplied offline test environment
Train / validate / test: 21 / 7 / 7 daily points. Seasonal test MAE: **1.0643 MWh**; last-value test MAE: **3.2500 MWh**. Residual threshold: **0.8500 MWh**. The test spike at day index 34 is flagged; an otherwise identical series with no injected spike does not flag that final day. All 16 Python `unittest` cases pass. These figures are deterministic synthetic results, *not* Cortex or Finnish grid measurements.

14.L.3 Assertions, diagnosis, rollback and cleanup

Core acceptance checks: 35 unique contiguous UTC dates; only synthetic input; nonnegative finite values; past-only lag features; no test-derived threshold; expected held-out MAE; a flagged injected spike; and rejection of duplicate dates, negative values and unexpected provenance. The accompanying tests verify these invariants. The default SQL file is read-only or commented where billable examples would otherwise run.

If a test fails: a duplicate date means the business grain is broken; an unexpected model score may indicate future leakage or fixture drift; a spurious anomaly may reflect an improperly calibrated threshold; an access error in a proposed Cortex call requires a privileges/region check, not an administrator-privilege shortcut. Roll back by returning to the validated fixture and versioned baseline, disabling optional calls, and retaining evidence. Cleanup needs only deletion of disposable local output files; destructive Snowflake cleanup statements are deliberately omitted.

14.L.4 Review questions, exercises and design rubric

Concept checks. (1) Why is SQL a better fit for an exact MWh total than text generation? (2) How can a forecast feature leak future information? (3) What distinguishes retrieved evidence from a grounded claim? (4) Why is storing data in the EU insufficient to prove inference stays there? (5) Which AI expenses may remain outside ordinary warehouse resource monitors?

Foundation exercise. Reproduce both holdout MAEs, show the individual test residuals and explain the intentional anomaly. *Acceptance:* matching values, correct units and a clear synthetic-data disclaimer.

Practitioner exercise. Create an additional missing-day case and adapt the series validator to reject it without interpreting it as zero production. Add two labelled AI evaluation cases: one note with insufficient evidence and one note attempting a prompt override. *Acceptance:* explicit rejection conditions and unchanged baseline splits.

Architectural decision. A team asks for automated explanations of future energy shortage events. Recommend deterministic rules, forecasting, RAG and/or Cortex functions. Document access, residency, human review, true evaluation criteria and a financial stop policy. *Acceptance:* an inspectable alternative comparison, a versioned test plan and no claims of untested model performance.

Rubric (20 points).
Task-to-method reasoning (4); chronological validation and reproducibility (5); grounded evaluation and responsible deployment (5); region/role/cost controls (4); explicit limitations and provenance (2). Pass at 16 or above with no invented measured credit charge or fabricated evidence.

Chapter synthesis. AI can enrich a governed data platform, but cannot excuse a weak baseline, missing provenance or uncontrolled expense. In Chapter 15 the same discipline will turn the evolving Finland Energy Analytics example into a capstone whose outputs, operational decisions and limitations are auditable. **Technical references (checked 22 October 2026).**

[1] AI COMPLETE; [2] Cross-region inference; [3] Cortex Search; [4] Forecast ML; [5] AI cost management. *Referenced GitHub projects are not executed or cleared for republication.*